KU LEUVEN

Chapter 4. Predictive Analytics: Nearest Neighbors and Ensembles

Data Science for Business

D0I74A

Prof. dr. Wouter Verbeke

Prof. dr. Bart Baesens



Course schedule

BLOCK1			
1	21/09/2026	Chapter 0. Course Outline Chapter 1. Data Science for Business	
1	22/09/2026	Chapter 1. Data Science for Business (ctd.)	
		Hands-on #1: Building a first pipeline	**Homework 1**
2	28/09/2026	Chapter 1. Data Science for Business (ctd.) Chapter 2. Exploratory Data Analysis & Preprocessing	
2	29/09/2026	Chapter 2. Exploratory Data Analysis & Preprocessing (ctd.)	
		Hands-on #2: EDA & Preprocessing	**Homework 2**
3	05/10/2026	Chapter 2. Exploratory Data Analysis & Preprocessing (ctd.)	
3	06/10/2026	Chapter 3. Predictive Analytics - Decision Trees	
4	12/10/2026	Chapter 4. Predictive Analytics - Nearest Neighbors & Ensembles	
4	13/10/2026	Chapter 5. Postprocessing	
5	19/10/2026	Chapter 5. Postprocessing	
		Hands-on #3: Postprocessing	**Homework 3**
5	20/10/2026	Hands-on #4: Assignment 1	Assignment 1

Outline

1. K-Nearest Neighbors
2. Ensembles
3. Bagging
4. Boosting
5. Random Forests
6. Bias and Variance

Learning objectives

- At the end of this chapter, students should be able to:
  - Understand and explain the workings of **nearest neighbors algorithms**
  - Understand and explain the workings of **ensemble algorithms**
  - Discuss the **use**, **advantages** and **disadvantages** of ensemble methods and kNN
  - Understand and explain the **bias-variance trade-off** and how it relates to different methods and measures that are discussed in this course, including ensemble methods

1. K-Nearest Neighbors

kNN

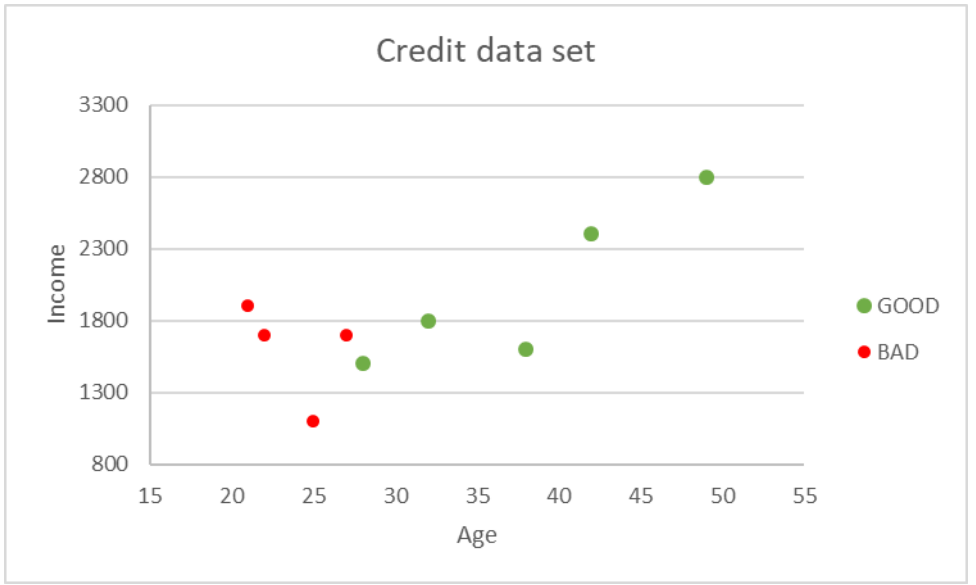
K-Nearest Neighbors

- Decision trees **segment instances**
  - Instances in a (leaf) node are ***similar***
    - In terms of their characteristics
    - In terms of their class or outcome value (that is, at least, the objective)
- As an alternative approach towards classification, we may more directly make use of ***similarity***
  - Similarity is used frequently, for a range of **tasks** and by a range of **methods**
  - Classification and regression (trees, cf. Chapter 3)
  - Clustering or segmentation (cf. Chapter 8): similar groups in customer base
  - Recommender systems: "*people who like X also like Y*"

K-Nearest Neighbors

- **Similarity**
  - **Entities** (e.g., customers) described by a **vector of variables** (e.g., *age*, *amount*) have a **position** in the attribute (input, variable or feature) space

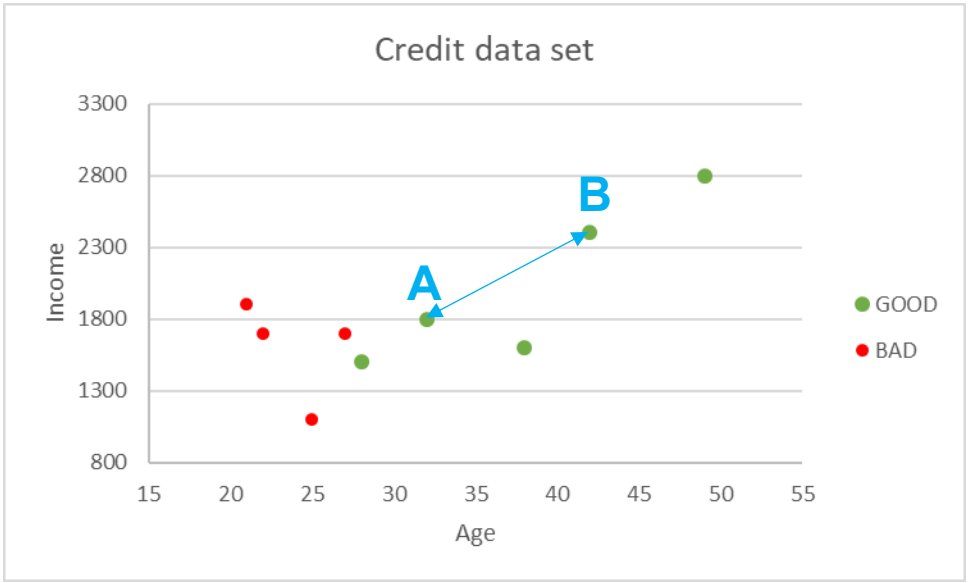
ID	Age	Income	Duration	…	Default
42882	25	1100	1	…	Bad
67088	21	1900	0	…	Bad
58922	22	1700	5	…	Bad
42440	27	1700	15	…	Bad
12580	42	2400	22	…	Good
24580	49	2800	37	…	Good
31008	32	1800	6	…	Good
21007	38	1600	0	…	Good
45880	28	1500	12	…	Good
…	…	…	…	…	…



K-Nearest Neighbors

- **Similarity**
  - The distance between two objects in variable space can be **measured**
    - E.g.: by means of Euclidean distance

	ID	Age	Income	Duration	...	Default
	42882	25	1100	1	...	Bad
	67088	21	1900	0	...	Bad
	58922	22	1700	5	...	Bad
	42440	27	1700	15	...	Bad
B	12580	42	2400	22	...	Good
	24580	49	2800	37	...	Good
A	31008	32	1800	6	...	Good
	21007	38	1600	0	...	Good
	45880	28	1500	12	...	Good
	...	...	...	...	...	...



K-Nearest Neighbors

Or, including duration in **3-D space**:

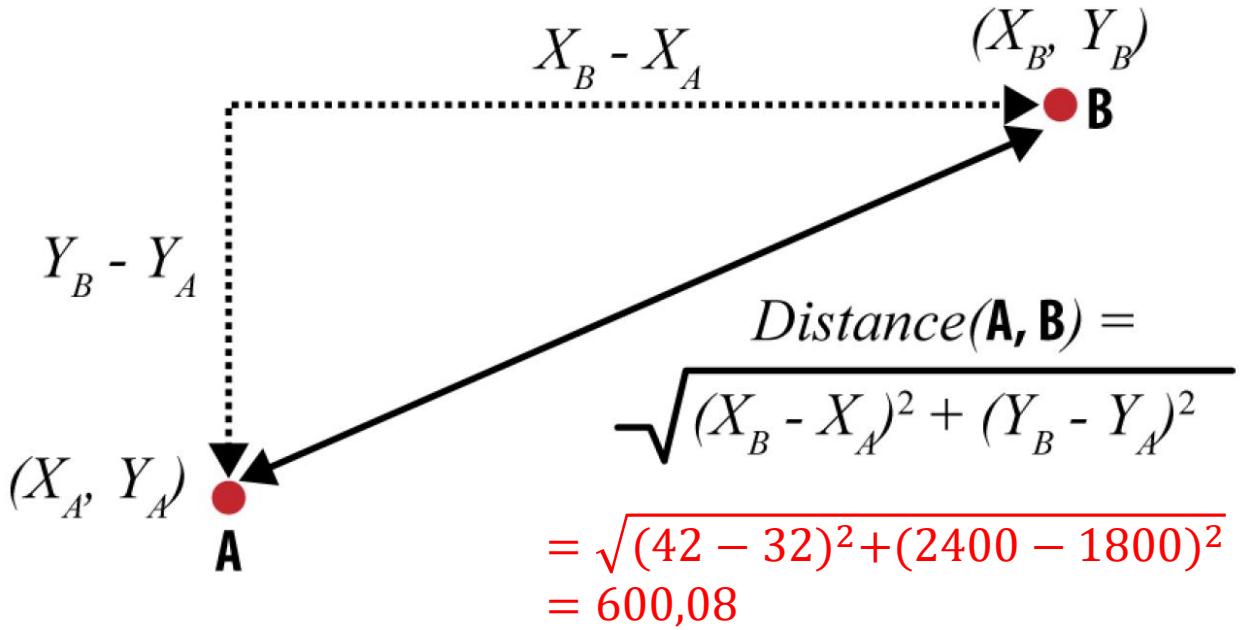
$$\text{Distance(A,B)} = \sqrt{(42-32)^2+(2400-1800)^2+\mathbf{(22-6)^2}}$$

Or, including **n variables** in **n-D space**:

$$\text{Distance(A,B)} = \sum_{i=1}^{n}\sqrt{(X_{i,B}-X_{i,A})^2}$$

- **Similarity**
  - The distance between two objects in variable space can be **measured**
    - E.g.: by means of Euclidean distance

	ID	Age	Income	Duration	…	Default
	42882	25	1100	1	…	Bad
	67088	21	1900	0	…	Bad
	58922	22	1700	5	…	Bad
	42440	27	1700	15	…	Bad
B	12580	42	2400	22	…	Good
	24580	49	2800	37	…	Good
A	31008	32	1800	6	…	Good
	21007	38	1600	0	…	Good
	45880	28	1500	12	…	Good
	…	…	…	…	…	…



Provost, F., & Fawcett, T. (2013). *Data Science for Business: What you need to know about data mining and data-analytic thinking*. O'Reilly Media, Inc., pp.308-311

K-Nearest Neighbors

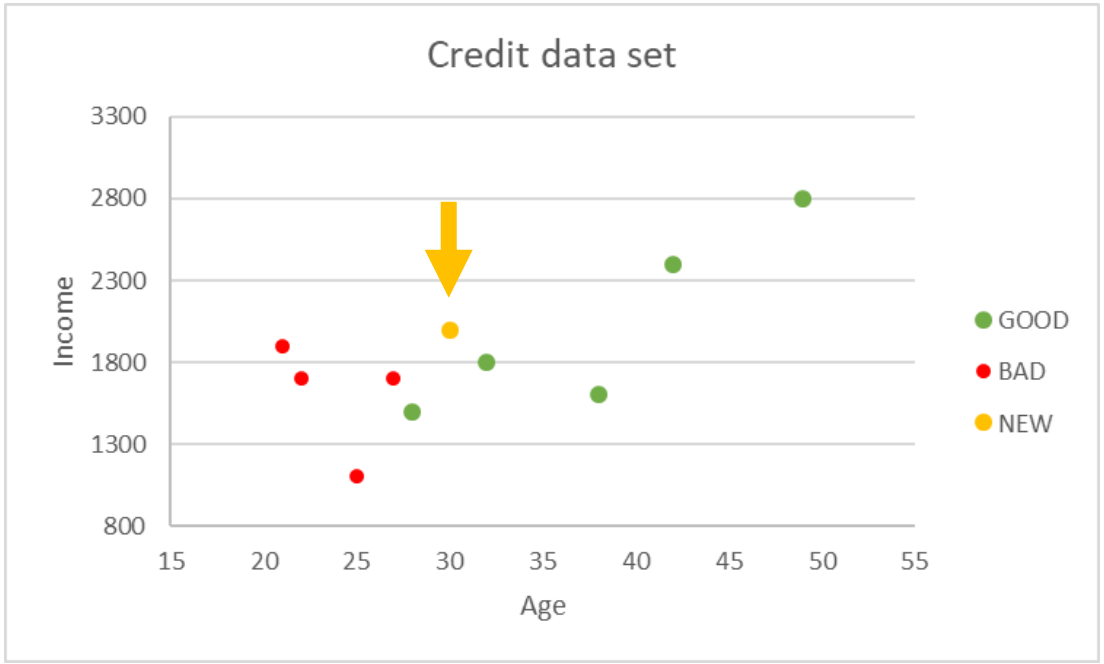
- **Similarity**
  - The distance between two objects in variable space can be **measured**
    - E.g.: by means of Euclidean distance
  - The smaller the **distance**, the more **similar** two objects

						Distance from:								
ID	**Age**	**Income**	**Duration**	**…**	**Default**	42882	67088	58922	42440	12580	24580	31008	21007	45880
42882	25	1100	1	…	Bad									
67088	21	1900	0	…	Bad	800,01								
58922	22	1700	5	…	Bad	600,01	200,00							
42440	27	1700	15	…	Bad	600,00	200,09	5,00						
12580	42	2400	22	…	Good	1.300,11	500,44	700,29	700,16					
24580	49	2800	37	…	Good	1.700,17	900,44	1.100,33	1.100,22	400,06				
31008	32	1800	6	…	Good	700,03	100,60	100,50	100,12	600,08	1.000,14			
21007	38	1600	0	…	Good	500,17	300,48	101,27	100,60	800,01	1.200,05	200,09		
45880	28	1500	12	…	Good	400,01	400,06	200,09	200,00	900,11	1.300,17	300,03	100,50	

K-Nearest Neighbors

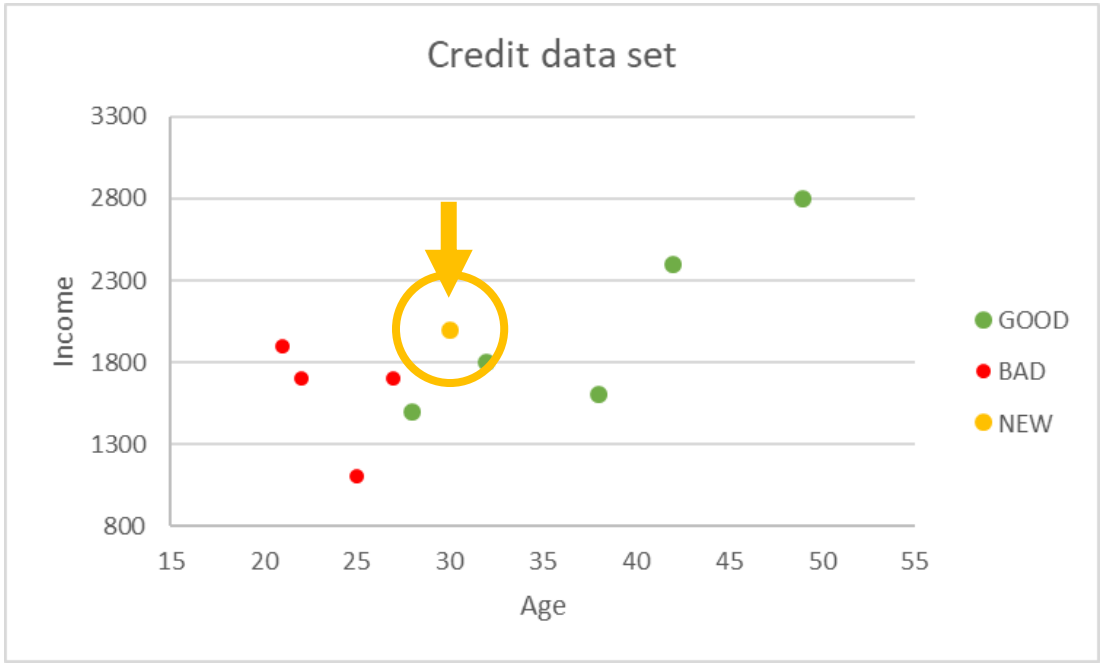
- Classifying ***unseen instances***?

ID	Age	Income	Duration	…	Default
42882	25	1100	1	…	Bad
67088	21	1900	0	…	Bad
58922	22	1700	5	…	Bad
42440	27	1700	15	…	Bad
12580	42	2400	22	…	Good
24580	49	2800	37	…	Good
31008	32	1800	6	…	Good
21007	38	1600	0	…	Good
45880	28	1500	12	…	Good
Unseen instance					
46890	30	2000	6	…	?



K-Nearest Neighbors

- Classifying ***unseen instances****?*
  - For classifying an object,
  - We can look at the observed class of the most similar objects in a database with historical records
  - i.e., the **nearest neighbors**,
  - and assign a class or class probability *based on* the observed class labels of these nearest neighbors (e.g., majority vote).
- Note: we don't use the class label in calculating similarity

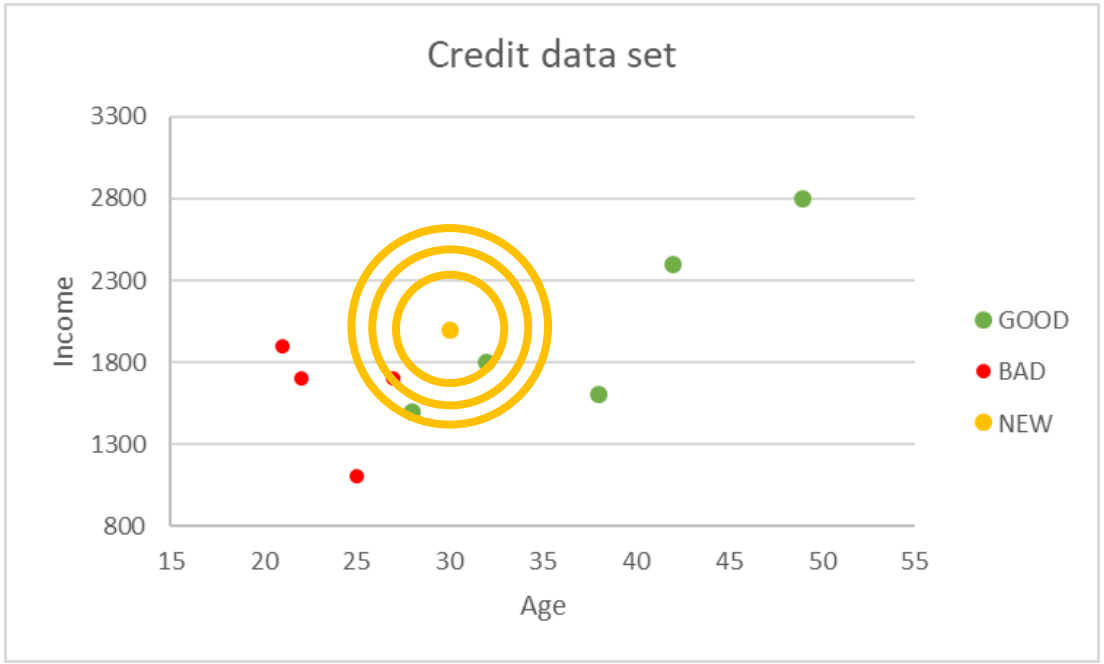


K-Nearest Neighbors

- **How many nearest neighbors?**
  - 1? 2? 3? 10? 100?
  - Related to size data set?
  - Odd number for breaking ties (with majority voting)
  - "k"

  → k-nearest neighbors (kNN)

  → **hyperparameter** that can be tuned
- Note: no model is trained or learned!
  - Model **IS** the historical data
- For regression: similar approach
  - E.g., use average observed outcome



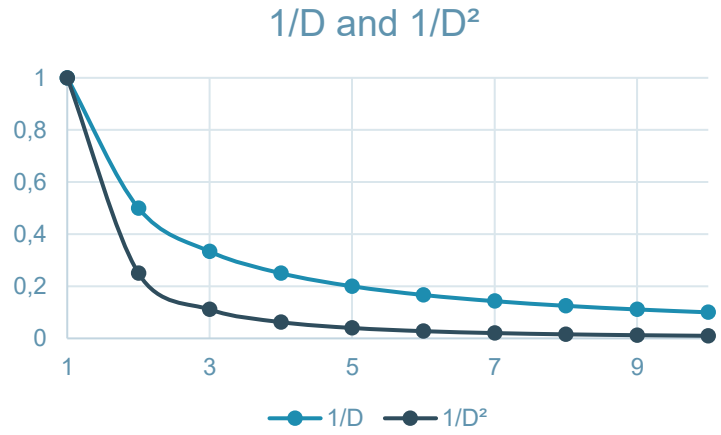
K-Nearest Neighbors

Higher k eventually leads to GOOD …
because there's more goods in the dataset!

Credit data set						Distance from:								
ID	**Age**	**Income**	**Duration**	**…**	**Default**	42882	67088	58922	42440	12580	24580	31008	21007	45880
42882	25	1100	1	…	Bad									
67088	21	1900	0	…	Bad	800,01								
58922	22	1700	5	…	Bad	600,01	200,00							
42440	27	1700	15	…	Bad	600,00	200,09	5,00						
12580	42	2400	22	…	Good	1.300,11	500,44	700,29	700,16					
24580	49	2800	37	…	Good	1.700,17	900,44	1.100,33	1.100,22	400,06				
31008	32	1800	6	…	Good	700,03	100,60	100,50	100,12	600,08	1.000,14			
21007	38	1600	0	…	Good	500,17	300,48	101,27	100,60	800,01	1.200,05	200,09		
45880	28	1500	12	…	Good	400,01	400,06	200,09	200,00	900,11	1.300,17	300,03	100,50	
Unseen instance														
46890	30	2000	6	…	?	900,01	100,40	300,11	300,01	400,18	800,23	200,01	400,08	500,00

		Similarity ranking:	9	1	4	3	6	8	2	5	7	majority voting		
		Class:	Bad	Bad	Bad	Bad	Good	Good	Good	Good	Good	Votes GOOD	Votes BAD	PRED.
k:		1		Bad								0	1	BAD
		2		Bad					Good			1	1	PAR
		3		Bad		Bad			Good			1	2	BAD
		4		Bad	Bad	Bad			Good			1	3	BAD
		5		Bad	Bad	Bad			Good	Good		2	3	BAD
		6		Bad	Bad	Bad	Good		Good	Good		3	3	PAR
		7		Bad	Bad	Bad	Good		Good	Good	Good	4	3	GOOD
		8		Bad	Bad	Bad	Good	Good	Good	Good	Good	5	3	GOOD
		9	Bad	Bad	Bad	Bad	Good	Good	Good	Good	Good	5	3	GOOD

K-Nearest Neighbors



- Every neighbour as much impact in voting/classifying?
- Similarity/distance may be used for **weighting the votes**
  - E.g., **weight**: reciprocal of the square of the distance → Reciprocal of $D^2 = 1/D^2$
  - E.g., **contribution**: proportional to the weight, but adding up to one → Then you get a ***score*** in [0,1]

Credit data set

ID	Age	Income	Duration	...	Default
42882	25	1100	1	...	Bad
67088	21	1900	0	...	Bad
58922	22	1700	5	...	Bad
42440	27	1700	15	...	Bad
12580	42	2400	22	...	Good
24580	49	2800	37	...	Good
31008	32	1800	6	...	Good
21007	38	1600	0	...	Good
45880	28	1500	12	...	Good
Unseen instance					
46890	30	2000	6	...	?

Weighted voting

Distance	Reciprocal D^2	Contribution
900,01	0,000001	0,01
100,40	0,000099	0,60
300,11	0,000011	0,07
300,01	0,000011	0,07
400,18	0,000006	0,04
800,23	0,000002	0,01
200,01	0,000025	0,15
400,08	0,000006	0,04
500,00	0,000004	0,02
Sum contrib. of bad instances:		**0,74**
Sum contrib. of good instances:		**0,26**

Classify unseen instance as BAD

K-Nearest Neighbors

- **Note**:
  - kNN relies on the use of ***some distance measure***
    - There's many (Euclidean, Manhattan, Mahalanobis, …), so **which to use?**
  - What about **categorical variables?**
    - Euclidean distance: only continuous variables
  - Sensitive to scale?
    - Euclidean distance: yes!
    - Need for standardization or normalization (cf. Chapter 2: Preprocessing)
    - E.g., running example:
      - Income ***overpowers*** age
      - I.e., has more importance because it is measured on a larger scale

K-Nearest Neighbors

- **Strenghts**
  - No distribution assumptions
  - Versatility: both classification and regression
  - Fast training phase (lazy learner)
  - Simple and easy to implement
  - Adapts easily to new data

K-Nearest Neighbors

- **Weaknesses**
  - Slow prediction phase (high computation cost)
  - High memory usage
  - Curse of dimensionality
  - Sensitive to feature scale
  - Sensitive to noise, outliers, and irrelevant features
  - Hyperparameter sensitivity

  - Often not so very powerful in predicting, yet, often used as a baseline

2. Ensembles

“k-models”

Ensembles

- Basic idea:
  - To improve predictive power
  - Instead of using one model
  - Could we use multiple models, e.g., two?
- E.g.: two-stage model (Van Gestel, 2005)
  - First model: interpretable white-box model, e.g.: logistic regression
  - Second model:
    - Predicts error made by first model
    - Advanced, powerful, non-linear black-box model
    - E.g.: Neural network, SVM , …

Van Gestel T. et al., 2005: Linear and non-linear credit scoring by combining logistic regression and support vector machines, Journal of Credit Risk

Two-stage model

Customer	Age	Income	Gender	...	Response
Emma	28	1000	F		No
Will	44	1500	M		Yes
Dan	30	1200	M		No
Bob	58	2400	M		Yes

Customer	Age	Income	Gender	...	Response	Logistic regression output
Emma	28	1000	F		No (=0)	0,44
Will	44	1500	M		Yes (=1)	0,76
Dan	30	1200	M		No (=0)	0,18
Bob	58	2400	M		Yes(=1)	0,88

Customer	Age	Income	Gender	...	Response	Logistic regression output	Error
Emma	28	1000	F		No (=0)	0,44	-0,44
Will	44	1500	M		Yes (=1)	0,76	0,24
Dan	30	1200	M		No (=0)	0,18	-0,18
Bob	58	2400	M		Yes(=1)	0,88	0,12

Customer	Age	Income	Gender	...	Logistic regression output	NN output	Final output
Bart	28	1000	F		0,68	-0,32	0,36

Step 1: Start from the original data.

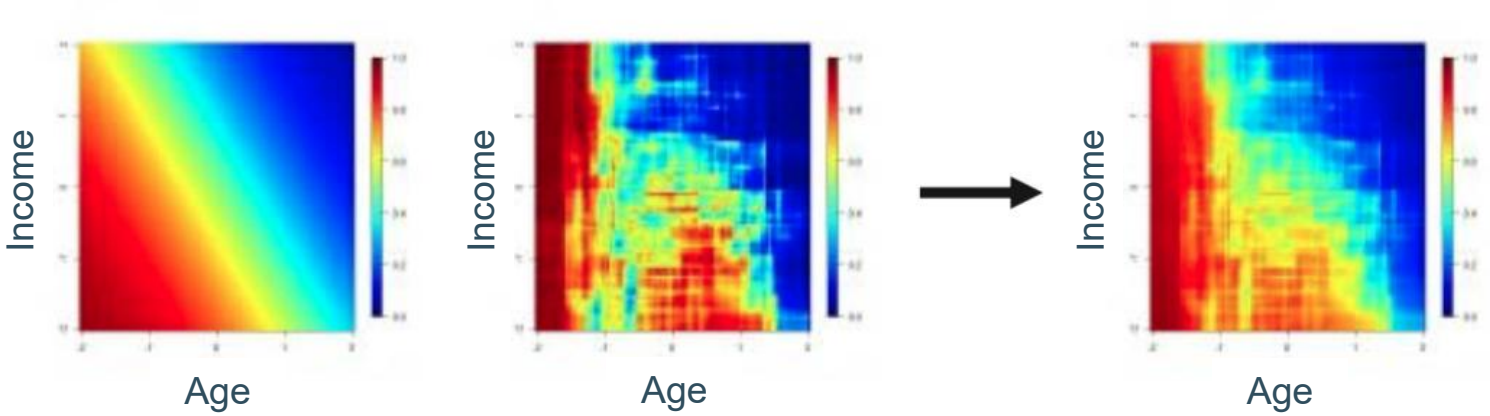
Step 2: Build a logistic regression model.

Step 3: Calculate errors from the logistic regression model.

Step 4: Build neural network predicting the errors from the logistic regression model.

Step 5: Score new observations by adding logistic regression and NN estimates.

Two-stage model





- Instead of two models, can we use three models?
- Four models?
- …

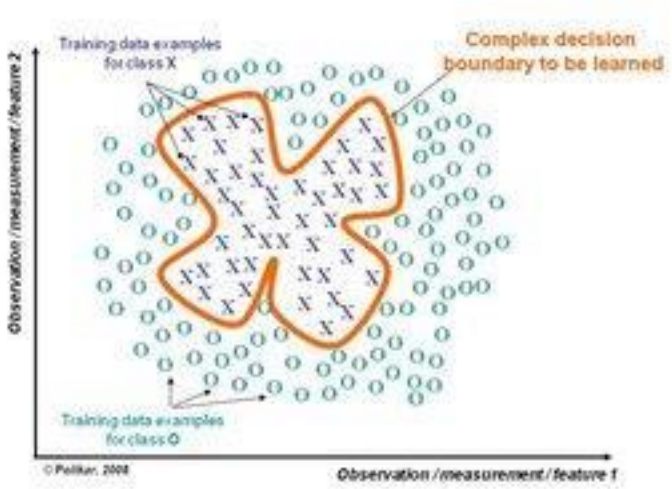
Ensembles

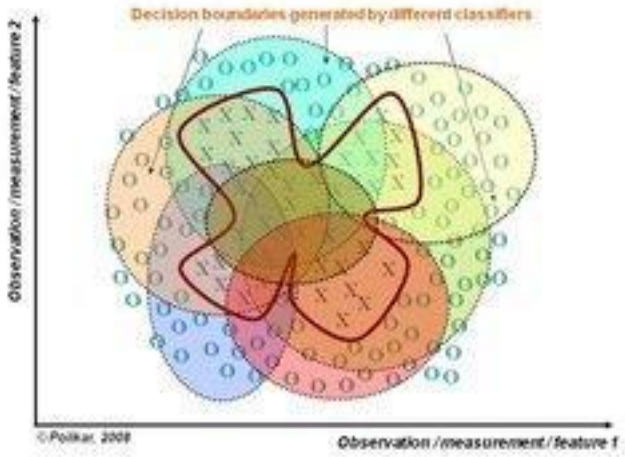
Similar to *k instances* in kNN, each casting a vote, we may build *k models*, each casting a vote

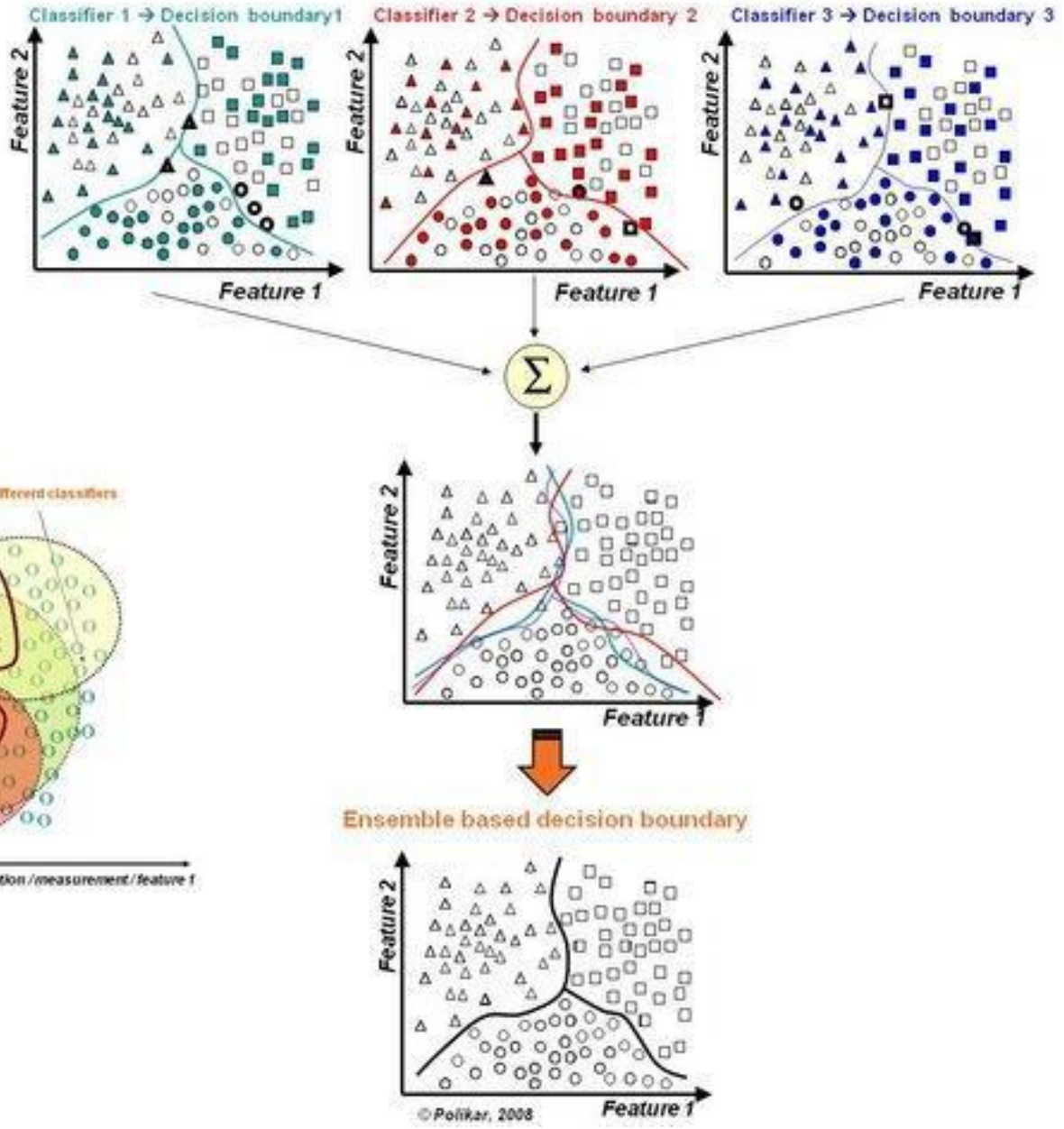
- Instead of using a single model, ensemble methods develop a ***series of models***
- To arrive at a final estimate, they combine the estimates
  - Classifier ***ensembles***
  - Ensemble = French for 'together'
- Why?
  - Wisdom of crowds idea: to arrive at better estimates
  - Different models may ***extract different relations & patterns from the data***
  - I.e., capture various bits and pieces of information in the dataset
- Different setups in combining,
  - E.g.: Bagging, Boosting, and Random Forests → smart, smarter, smartest

Ensembles

- From a single model to an ensemble:





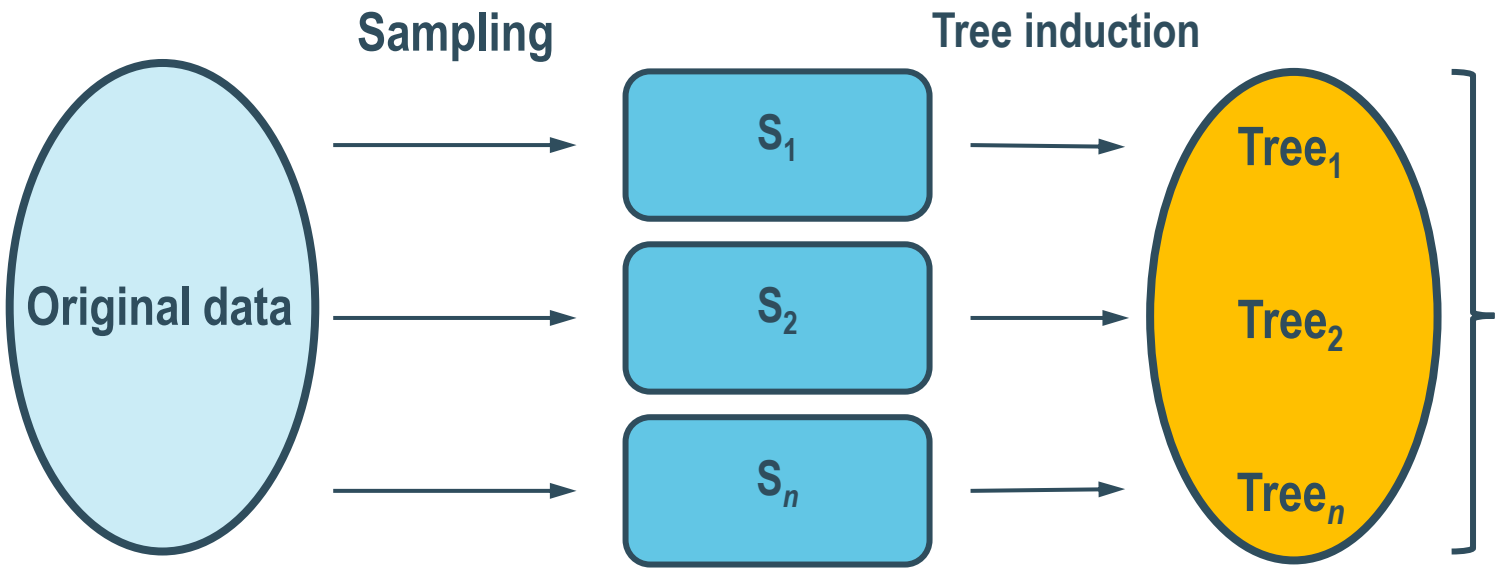


http://www.scholarpedia.org/article/Ensemble_learning

Ensembles

- How to generate models that '***extract different relations and patterns from the data***'?
- Ensemble methods = **meta-learning schemes**
  - Can be combined with 'any' **base learner**
  - Most popular: decision tree

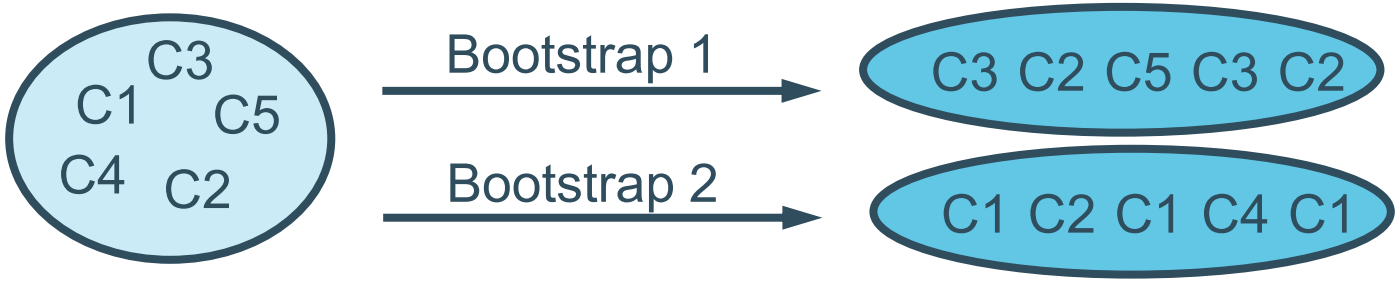
How to generate samples?
How to learn trees?
How to combine estimates?



$Classification = f(Tree_1, Tree_2, ..., Tree_n)$

Generating samples

- To obtain various models, the **samples** used to develop these models **should be as different as possible**:
  - 'Cross-validation' > folds *mwah*
  - Random subsample *mwah*
  - Bootstrapping: sampling with replacement *yes!*



- **Note**:
  - Ensemble learning 'requires' **unstable base-classifier**! **Why?**
  - Decision trees and decision stumps (one-level DT) most popular
  - Stable classifiers are less useful as base classifier (e.g., logistic regression, kNN, SVM)

Combining estimates

- Two-stage approach discussed before: additive setup

- Alternatives:
  - Combination rules
  - Voting schemes
  - Stacking
  - …

Combination rules

- Suppose we created two models
  - Model 1: logistic regression model
  - Model 2: decision tree

Optimized threshold for **maximizing accuracy**

True label	Model 1 (0.54)	Model 2 (0.50)	Ensemble?
Yes	0.80 (yes)	0.70 (yes)	
Yes	0.78 (yes)	0.50 (yes)	
Yes	0.54 (yes)	0.50 (yes)	
No	**0.57 (yes)**	0.30 (no)	
No	0.30 (no)	**0.70 (yes)**	
No	0.22 (no)	0.40 (no)	

Combination rules

- Suppose we created two models
  - Model 1: logistic regression model
  - Model 2: decision tree

With optimized threshold!

True label	Model 1 (0.54)	Model 2 (0.50)	Min (0.50)	Max (0.78)	Mean (0.52)
Yes	0.80 (yes)	0.70 (yes)	0.70 (yes)	0.80 (yes)	0.75 (yes)
Yes	0.78 (yes)	0.50 (yes)	0.50 (yes)	0.78 (yes)	0.64 (yes)
Yes	0.54 (yes)	0.50 (yes)	0.50 (yes)	0.54 (no)	0.52 (yes)
No	0.57 (yes)	0.30 (no)	0.30 (no)	0.57 (no)	0.44 (no)
No	0.30 (no)	0.70 (yes)	0.30 (no)	0.70 (no)	0.50 (no)
No	0.22 (no)	0.40 (no)	0.22 (no)	0.40 (no)	0.31 (no)

Voting schemes

- Majority voting
  - Model 1 votes yes
  - Model 2 votes yes
  - Model 3 votes no
  - Model 4 votes no
  - Model 5 votes yes
  - Model 6 votes yes

  - … yes wins, 4 to 2

Voting schemes

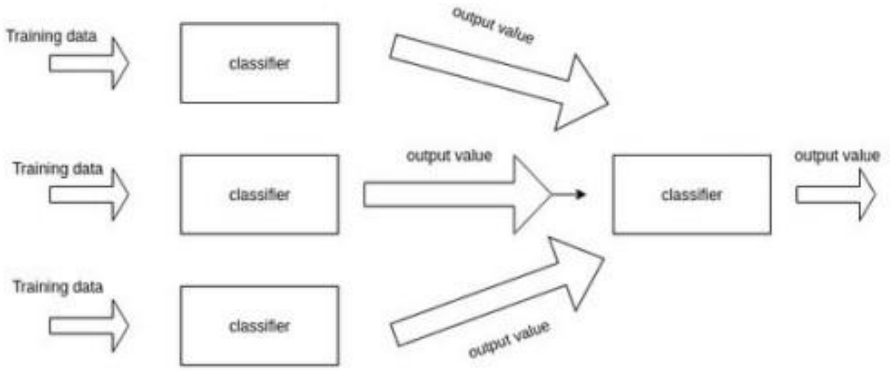
- **Weighted** majority voting
  - Model 1 votes yes x 1
  - Model 2 votes yes x 2
  - Model 3 votes no x 4
  - Model 4 votes no x 6
  - Model 5 votes yes x 1
  - Model 6 votes yes x 1

  - … no wins, 10 to 5
- Setting the **optimal weights** may be a training step in itself!

Cf. K-Nearest Neighbors!

Stacking

- Models can be combined in other manners as well:
  - A **first level of models** uses the 'raw inputs' as predictors,
  - A **second level model** can be trained using as inputs the predictions of a series of first level models: **stacking**
    - I.e., the estimates of an ensemble of models are used as inputs to a second level model, **instead of** used as input to a **voting scheme**



Automated feature engineering!

https://www.kdnuggets.com/2016/11/data-science-basics-intro-ensemble-learners.html

3. Bagging

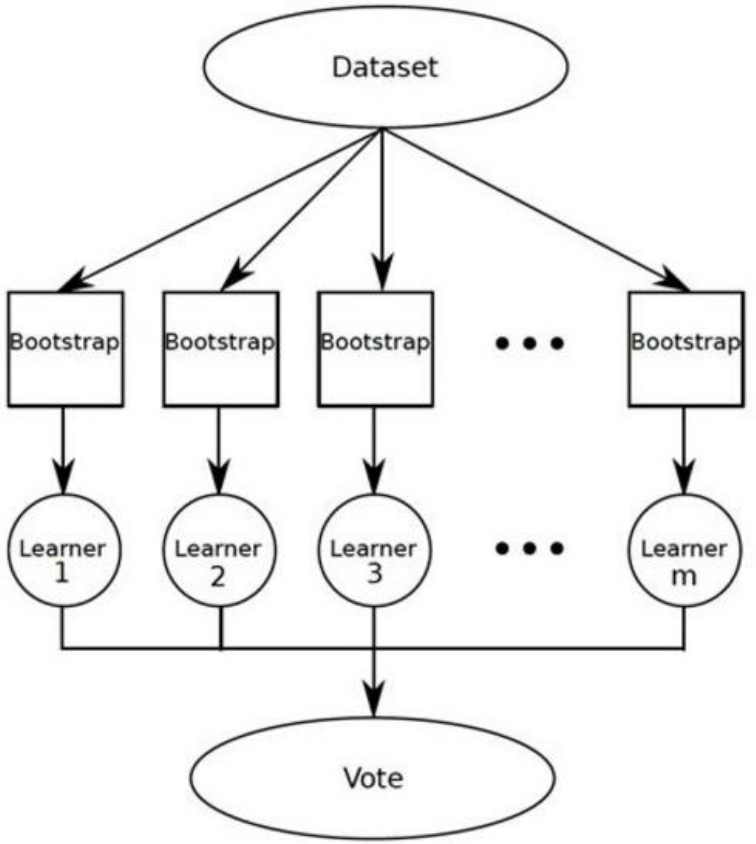
Bagging

- Bootstrap aggregating
- Take *N* bootstraps from the original data sample.
  - *N* = ~30, or tune via validation set (hyperparameter!)
- Build a classifier (decision tree) for each bootstrap.
  - **For Classification:**
    - A new observation is classified by letting all *N* classifiers **vote** ((weighted) majority voting scheme, ties resolved arbitrarily).
  - **For Regression:**
    - Prediction is the **average** of the outcome of the various regression trees.
- "*The vital element is the **instability** of the prediction method. If perturbing the learning set can cause significant changes in the predictor constructed, then bagging can improve accuracy.*" (Breiman, 1996)

Bagging

- Final estimate: **simple majority voting**

- Note:
  - Individual classifiers are learned **independently**!
  - Bootstrap samples are drawn independently
  - Fast implementation possible in parallel setup



4. Boosting

Boosting

- Train multiple classifiers using a **weighted sample of the training data**
- Iteratively **reweight the training distribution** according to the **classification error** (or other loss function)
  - Misclassified cases get a higher weight
  - Correctly classified cases get a lower weight
  - Difficult observations iteratively receive more attention!
- Either the classification algorithm can directly work with **weighted training examples**, or if not, **sample** a new training set according to the weight distribution
- The final model is a **weighted combination** of all individual classifiers (cf. Section 2)
- Several **variants**
  - Most popular: AdaBoost (Adaptive Boosting) (Freund and Schapire, 1997)

AdaBoost (Freund and Schapire, 1999)

- Given: (x_1,y_1), ..., (x_N, y_N) where x_i is the attribute vector and y_i {1,-1}
- Initialize $W_1(i) = 1/N$, $i = 1, \ldots, N$
- For $t = 1,...,T$
  - Train weak classifier (for example, decision tree) using weights W_t
  - Get weak classifier C_t with classification error ε_t
  - Choose $$\alpha_t = \frac{1}{2}\ln(\frac{1-\varepsilon_t}{\varepsilon_t})$$
  - Update $$W_{t+1}(i) = \frac{W_t(i)}{Z_t} \times \begin{cases} e^{-\alpha_t} & if\ C_t(x_i) = y_i \\ e^{\alpha_t} & if\ C_t(x_i) \neq y_i \end{cases}$$

    where Z_t is a normalization factor such that $$\sum_{i=1}^{N} W_{t+1}(i) = 1$$
- Output the final ensemble model
  - I.e.: a weighted voting scheme is used!

$$E(x) = sign(\sum_{t=1}^{T} \alpha_t C_t(x))$$

Boosting

- Multiple loss functions are possible for calculating the classification error
  - Misclassification, exponential, cross-entropy, squared error, …
- The α_t value measures the importance that is assigned to C_t, and increases as ε_t gets smaller
- As the algorithm proceeds, the weights tend to concentrate on 'hard' examples
- A stopping decision can be made using an independent validation set
- **Variants**:
  - AdaBoost.M1: multi-class classification
  - AdaBoost.M2: multi-class classification
  - AdaBoost.R1: regression
  - AdaBoost.RT: regression

Boosting

- Advantages
  - T is the only parameter to tune
  - Fast, simple, and easy to program

- Disadvantages
  - Boosting can have a risk of **overfitting** to the hard (=noisy) examples in the data

- Aternative: **Gradient Boosting**
  - Implements tree boosting whereby a tree in the ensemble is fit to the residual of the prediction from the earlier trees
  - **XGB**: eXtreme Gradient Boosting
  - **LightGBM**: light Gradient Boosting Machines

5. Random Forests

Random forests

- Introduced by Breiman (2001)
- Given a data set with *N* observations and *M* inputs
- *m* = constant chosen beforehand
- For *t* = 1,…,*T* (for example, *T* = 50, 100):
  - Take a bootstrap sample with *N* observations
  - For each node of a tree, **randomly** choose *m* inputs on which to base the splitting decision
  - Split on the best of this subset
  - Fully grow each tree without pruning
- Common choices for m: *m* = 1, *m* = 2, *m* = floor($\log_2(M)$+1) , …
  - Can be used for both classification and regression.
  - Also referred to as Random Forests RI (random input selection)

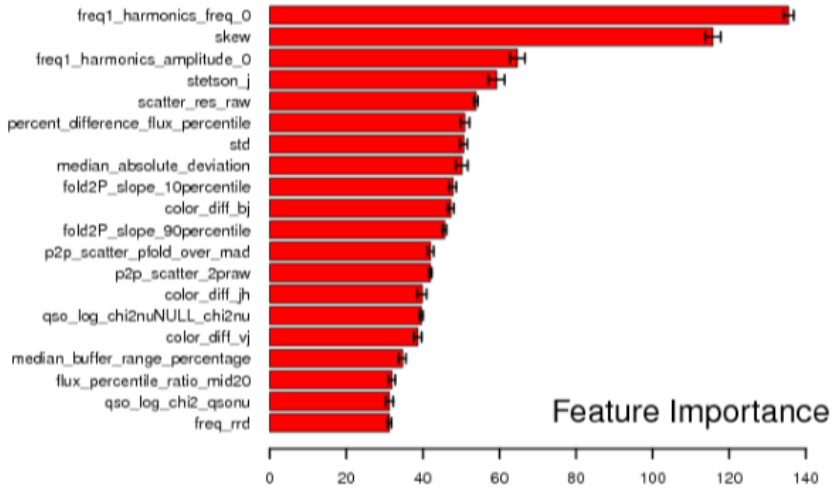
Random forests

- Advantages
  - Only two parameters to determine: *T* and *m*
  - Highly accurate, empirical evaluation shows that its accuracy is as good as AdaBoost and sometimes better (Breiman 2001)
  - Able to cope with a very large number of inputs
  - Faster than bagging or boosting because there are fewer inputs for each tree

- Disadvantages
  - Decreased interpretation because of multiple trees, as with any ensemble

Feature importance

- How to **explain 100 trees vs. 1**?
- **Feature importance**
  - Score for each feature indicating **how helpful a feature is**
  - In contributing to the predictive power of a model

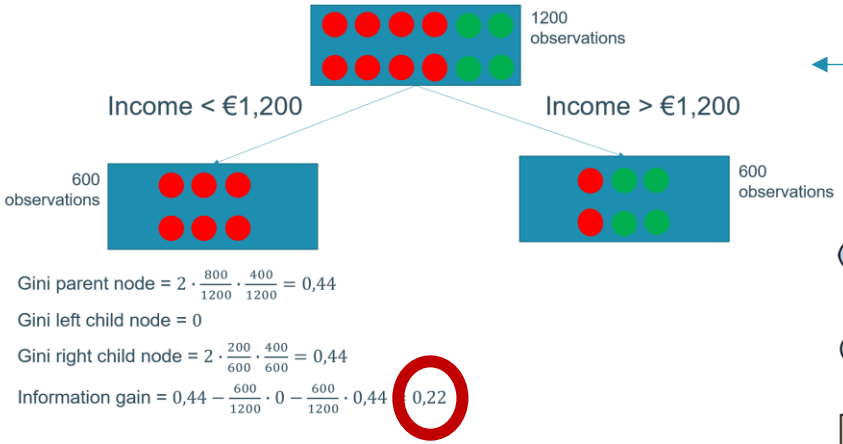


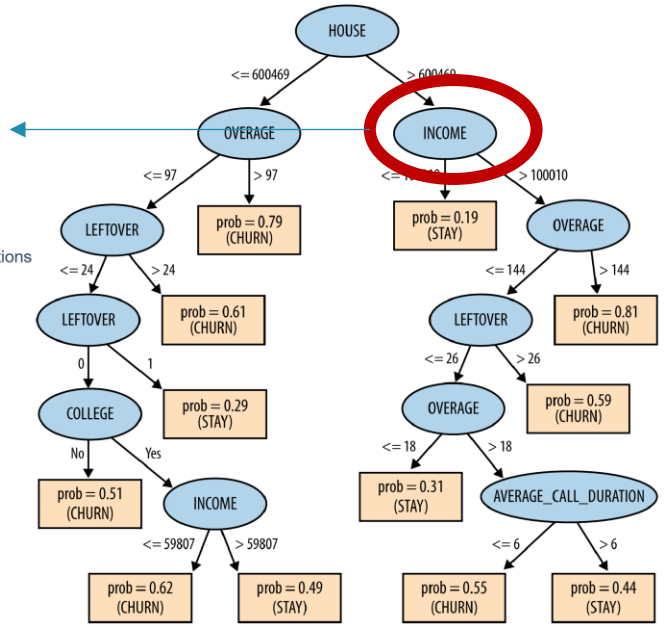
Feature importance

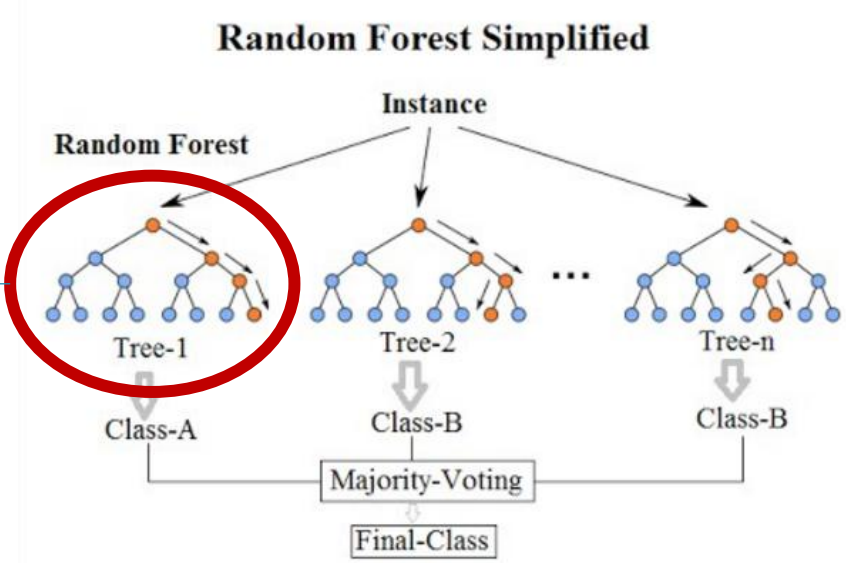
- **Step 1: Compute Node Importance**
  - = information gain in a node multiplied with the proportion of observations that reached that node
- **Step 2: Compute Feature Importance for a Single Tree**
  - = the sum of the node importances where feature k was used to split, divided by the sum of all node importances in that tree
- **Step 3: Ensemble Average Feature Importance**
  - = average of feature importance scores across all trees in the forest

Feature importance

- **Step 1: Compute Node Importance**
  - = information gain in a node multiplied with the proportion of observations that reached that node







E.g., if 50% of observations in this node:

- NI_income = 0,22 x 0,5 = 0,11

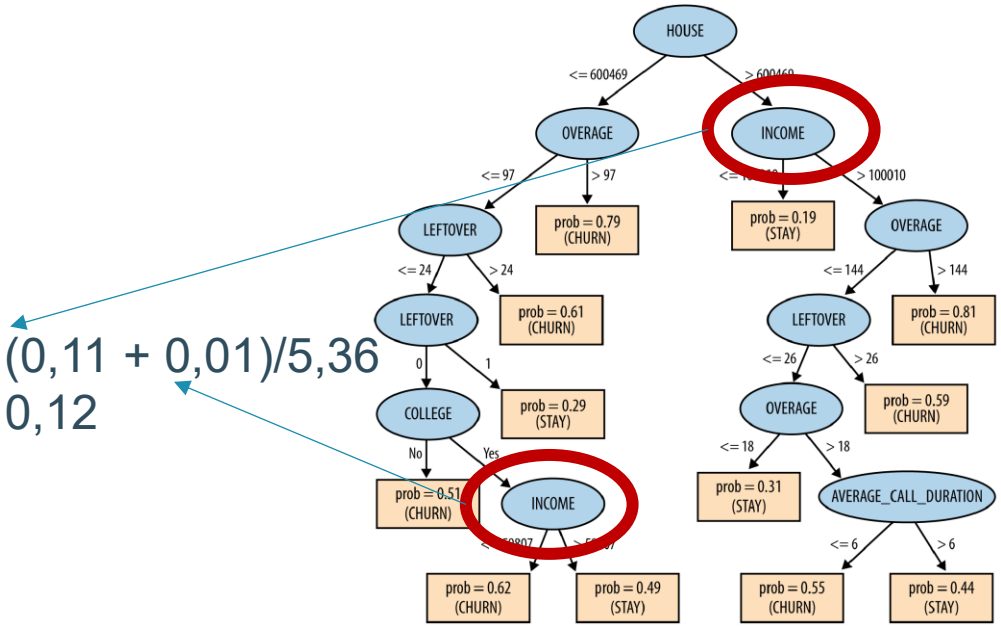
Feature importance

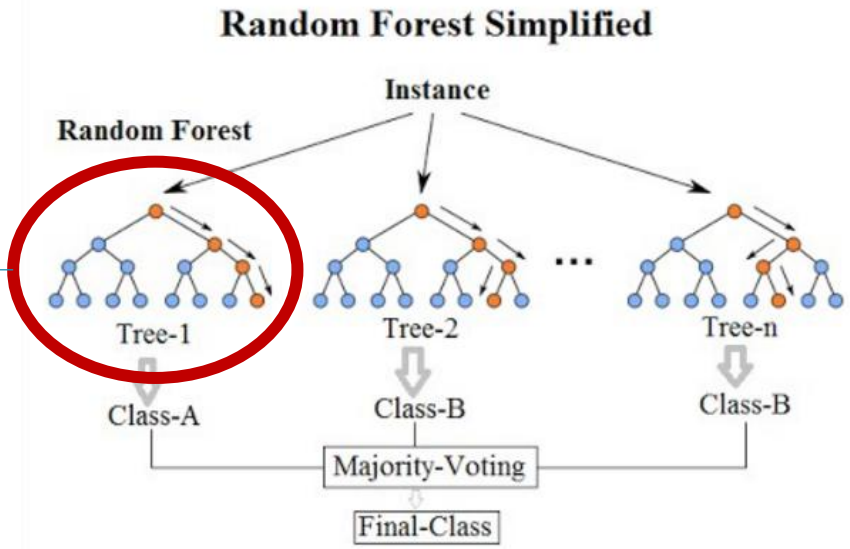
- **Step 2: Compute Feature Importance for a Single Tree**
  - = the sum of the node importances where feature k was used to split, divided by the sum of all node importances in that tree

E.g.,

- If NI_tree-1 = 5,36

- FI_income_tree-1 = (0,11 + 0,01)/5,36
  = 0,12



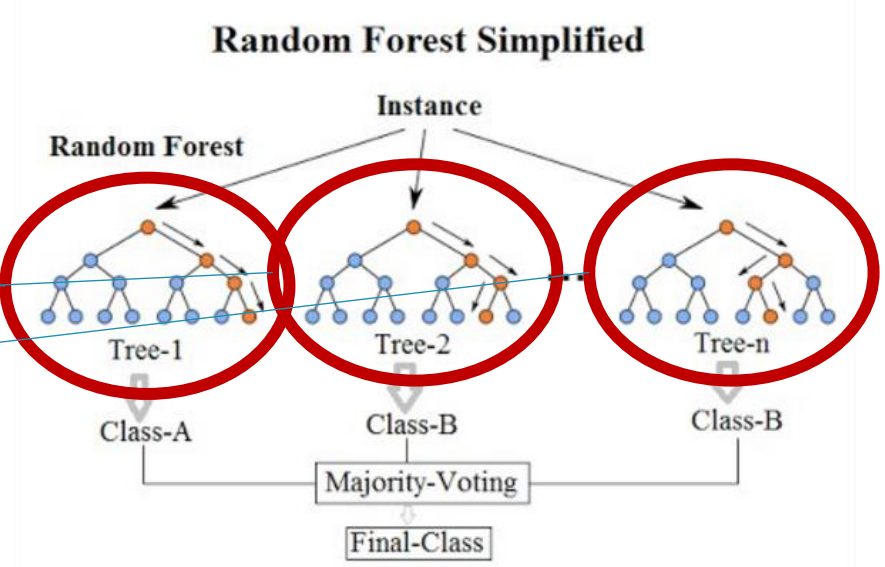


Feature importance

- **Step 3: Ensemble Average Feature Importance**
  - = average of feature importance scores across all trees in the forest

E.g.,
- FI_income_tree-1 = 0,12
- FI_income_tree-2 = 0,58
- …
- FI_income_tree-n = 0,42

- FI_income = (0,12 + 0,58 + … + 0,42)/n
  = 0,37



6. Bias and Variance … and Ensembles

What determines how *good* a model can be?

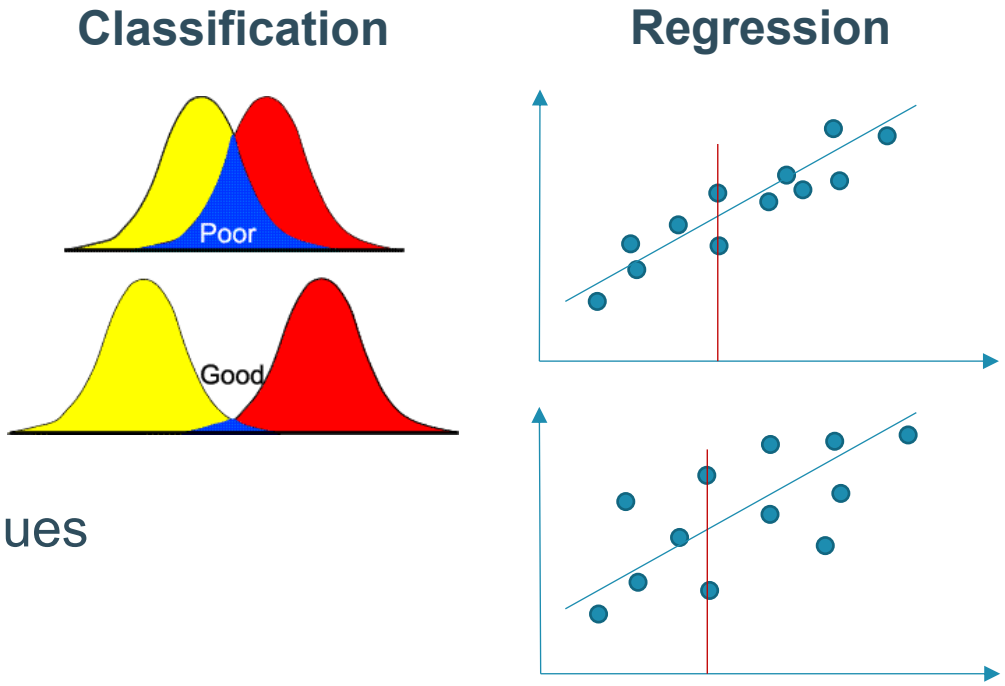


Bias and variance

- Errors by a model/method are characterized and determined by three factors:
  1. Inherent randomness
  2. Bias
  3. Variance

Provost, F., & Fawcett, T. (2013). *Data Science for Business: What you need to know about data mining and data-analytic thinking*. O'Reilly Media, Inc., pp.308-311

Bias and variance



1. **Inherent randomness**
   - Inherently probabilistic outcome:
     - Instances with exactly the same attribute values
     - Have different values for the target variable
     - Non-deterministic outcome

   - Hence, given the available data:
     → Maximum theoretical accuracy (= 1 – **Bayes' error rate**)
     → **Bayes' error rate** is the lowest possible error rate for a probabilistic classification problem
     → Less or more stochasticity?
       - Depends on the data generating process (& noise)

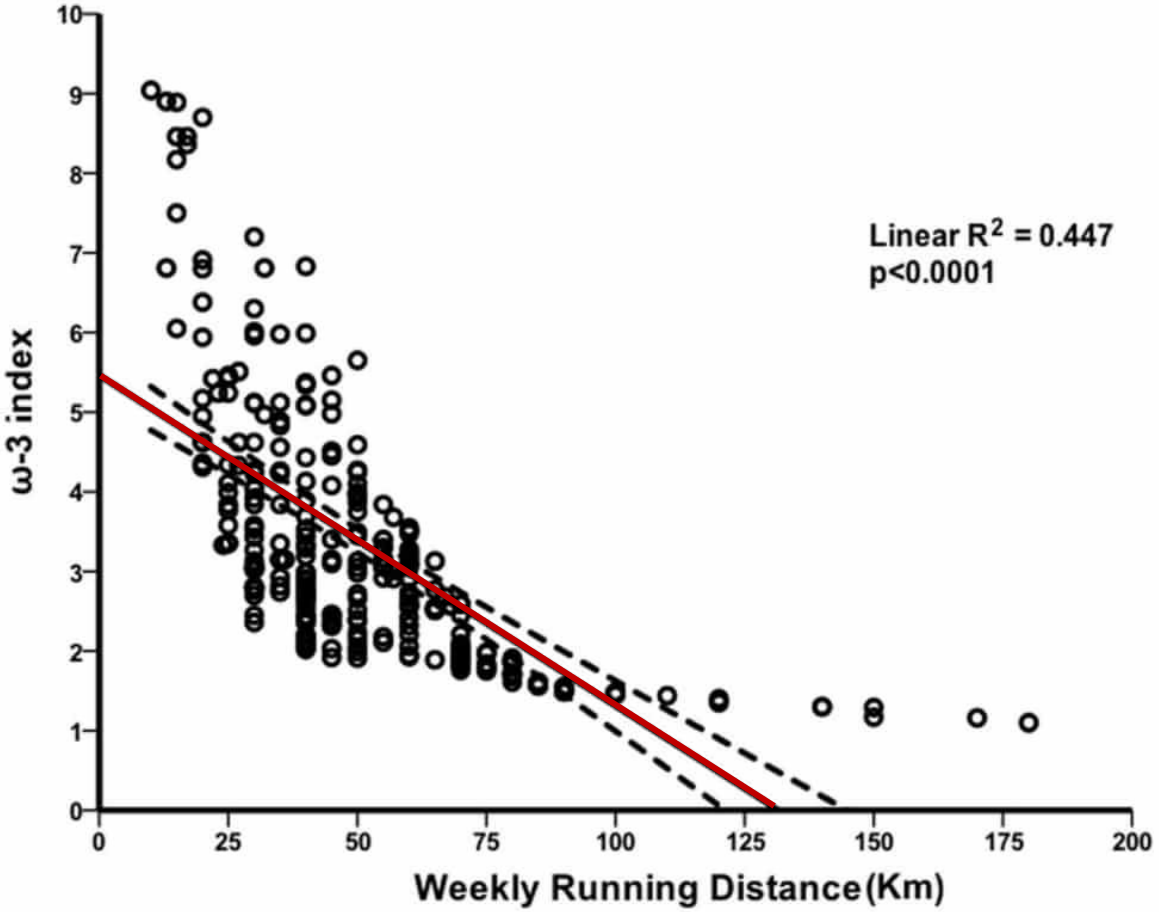
Provost, F., & Fawcett, T. (2013). *Data Science for Business: What you need to know about data mining and data-analytic thinking*. O'Reilly Media, Inc., pp.308-311

Bias and variance

The Omega-3 Index test is simply a measure of the amount of EPA and DHA in the blood, specifically the red blood cell membranes. For example, if you have 64 fatty acids in a cell membrane and 3 are EPA and DHA, then you would have an Omega-3 Index of 4.6%.

2. Bias

- Modeling procedure may be **biased**
  - The model that is fit to the data, may not fit the data (well)
  - E.g.: a linear model can never fit an exponential curve
    - 'Capacity' of the model
- A model therefore may not be able to reach the Bayes rate, no matter how much data is provided
  - **~Underfitting**

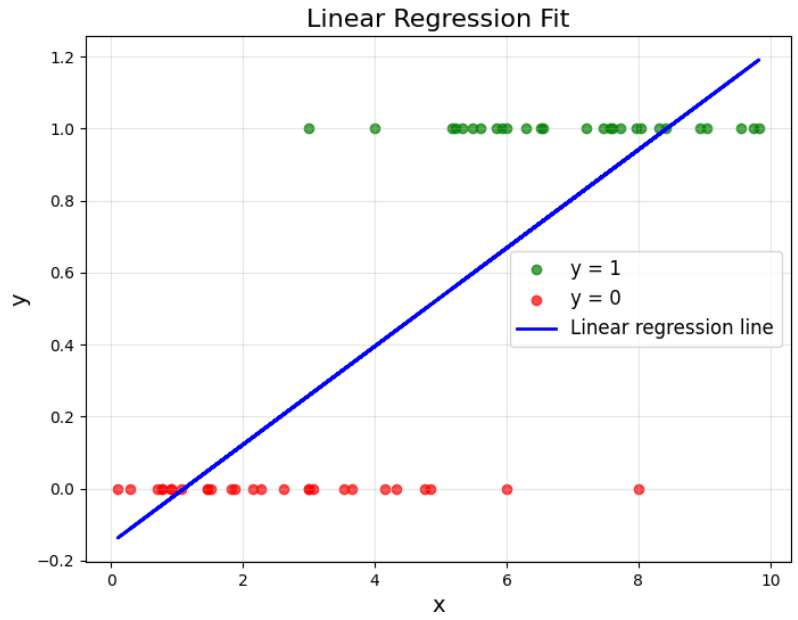


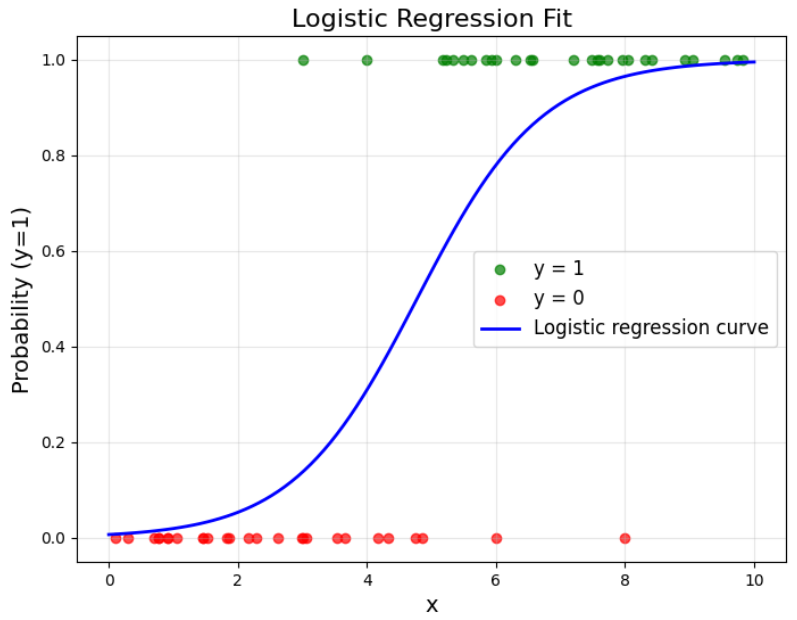
https://www.prorun.nl/gezondheid/het-onzichtbare-gevaar-van-veel-kilometers-per-week/
https://omegaquant.com/what-is-the-omega-3-index/

Bias and variance

2. Bias

- Linear regression to estimate a binary target variable? Biased model!





Bias and variance

What is a modeling method (aka modeling procedure)?
What is (the difference with) a model?

3. Variance

- We do not have an infinite amount of training data, we do not observe the (entire) population (e.g., future behavior of customer) → **we use a sample**
- Different models are obtained for (possibly even slightly) different samples, resulting in different models that achieve different "*accuracy*"
- The ***variability in accuracy*** across samples is called the **variance of a modeling method**
- We've encountered 'variance' before, but called it differently:

INSTABILITY!

- Note: variance may be the result from overfitting!

Bias and variance

What is a model? What is a modeling method?

Terminology!

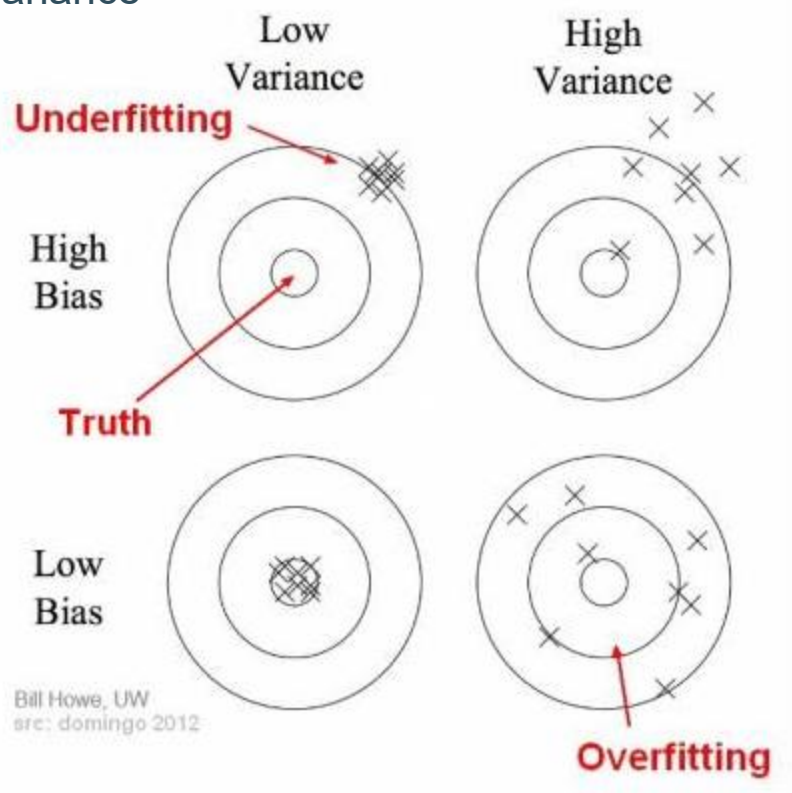
Bias and variance are characteristics of a model(-ing method), which impact the errors made by a model in estimating an outcome

- It depends!
  - Some (statisticians) call linear regression a model (a linear model). This model is **fitted** on data, where the **(fitting) method** is, e.g., OLS
  - Your professor (engineer/data scientist) calls linear regression, including the fitting procedure (OLS) a method, with the result when applied a data sample 'a model'
- Bias and variance are characteristics of (what I call) modeling methods, not of **a single**, **fitted model**, e.g.:
  - OLS linear regression is inherently biased in fitting nonlinear data
    - As a result, a fitted linear regression model may be biased (cf. omega-3 example)
  - Decision trees **inherently** have higher variance than logistic regression
    - They are inherently more 'unstable'

Bias and variance

- Preferably, we have a modeling procedure with no (or low) bias and no (or low) variance
  - However: we face **a bias-variance trade-off**!
  - Methods with low bias tend to have high variance
  - Methods with low variance tend to have high bias
- Extreme cases:
  - Predictive model = average churn rate:
    - Low variance, high bias
  - Linear regression with 1000 variables:
    - High variance, low bias
  - The 1000-variable model does not necessarily perform better: we don't know exactly which source of error (bias or variance) will dominate (for these models, for this problem)
    - Also **depends on the size of the dataset**

A model with no variance or very low variance means that you **minimally adjust or fit the model to the data**.
If you maximally fit the model to the data, then you possibly minimize the error by maximally leveraging the information (patterns, relations) in the data; but then, every change in the data results in a change in the model, yielding high variance

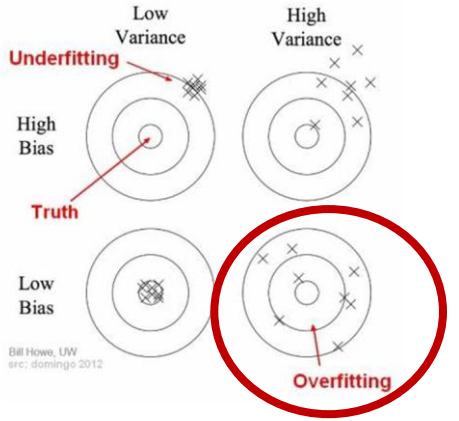


https://towardsdatascience.com/understanding-the-bias-variance-tradeoff-165e6942b229

Bias and variance

- The 1000 variable model:
  - Has many parameters to be fitted, i.e., is highly complex
    - Higher complexity typically gives more flexibility to fit the data
    - Higher complexity typically gives lower bias but higher variance
  - The 1000 variable model is likely **too complex**, i.e., is overfitting
  - A solution is to apply some **complexity control mechanism**
    - E.g., **selecting a subset of the variables** to use, resulting in less parameters to fit
  - Complexity control tries to manage the trade-off between bias and variance
    - To find a sweet-spot where the combination of the errors from each is smallest
  - Complexity control? E.g.:
    - Variable selection with AIC, BIC, …
    - Pruning decision trees
    - Regularization (cf. Chapter 6)
    - …

Bias and variance … and ensembles



- If we have a modeling method with **high variance**
- Then **averaging over multiple models or predictions reduces variance**
  - E.g., of the ensemble method compared to the single tree method

- Ensemble methods therefore *tend to* improve the predictive ability more for higher-variance methods
  - I.e., in cases where you would expect more overfitting
  - E.g., with decision trees
    - Can dramatically overfit the data
    - Are very sensitive to changes in the training data (cf. Chapter 3)

Friedman, J. H. (1997). On bias, variance, 0/1—loss, and the curse-of-dimensionality. *Data mining and knowledge discovery*, *1*(1), 55-77
https://towardsdatascience.com/understanding-the-bias-variance-tradeoff-165e6942b229

Thinking about errors…

- Case: Zillow home-flipping business
  - Zillow launched a business model where it used its algorithm (its “Zestimate” valuations) to buy homes, perform (minor) renovations, and then resell them.
  - The company publicly acknowledged that the algorithmic model failed because **it could not accurately predict future home prices in a highly volatile market**.
  - As a result, Zillow shut down its “iBuying” home-flipping business in November 2021, taking large losses.

- Error = Actual value – Estimated value
  - Positive error: model underestimated the price
  - Negative error: model overestimated the price

- Effect and importance of overestimation vs. underestimation?



https://slate.com/technology/2021/11/zillow-house-flipping-failure-awesome.html

Q&A